



Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas

Abstracción y uso de datos

REPORTE

Yventz Entzana Garduño

Maya García Aldo
N.L. 24

Martinez Perez Alan Eduardo
N.L. 22

3CM3O

1. Ordenador.h

Es una clase base genérica que proporciona la estructura común para los algoritmos de ordenamiento.

Funcionalidad:

- Almacena un arreglo de datos.
- Guarda el tamaño del arreglo.
- Permite imprimir los elementos.

En este caso aplicamos programación orientada a objetos, herencia, plantillas y encapsulamiento.

2. BubbleSort.h

Implementamos el algoritmo de ordenamiento Burbuja (bubble sort).

Funcionalidad:

- Compara elementos adyacentes.
- Intercambia posiciones si están desordenados.
- Repite el proceso hasta ordenar.

Aplicamos conceptos como herencia, uso de swap e iteración con ciclos.

3. QuickSort.h

Implementamos el algoritmo Quick Sort, basado en el uso de un pivote.

Funcionalidad:

- Selecciona un pivote.
- Divide el arreglo en menores y mayores.
- Ordena recursivamente.

Aplicamos conceptos de recursividad, división de problemas y herencia.

4. MergeSort.h

Implementamos el algoritmo de Merge Sort.

Funcionalidad:

- Divide el arreglo en partes.
- Ordena cada parte.
- Fusiona los resultados.

Aplicamos conceptos de recursividad, uso de arreglos auxiliares y herencia.

5. BubbleIndirecto.h

Funcionalidad:

- Usa un arreglo de punteros.
- Ordena las direcciones de memoria.
- No modifica los datos originales.

Aquí aplicamos conceptos de punteros y manejo indirecto de datos.

6. QuickIndirecto.h

Versión indirecta del algoritmo Quick Sort.

Funcionalidad:

- Usa punteros para ordenar.
- Aplica partición y recursividad.

Aplicamos punteros dobles y recursividad.

7. MergeIndirecto.h

Funcionalidad:

- Divide y fusiona punteros.
- Mantiene intactos los demás originales.

Aplicamos punteros, recursividad y manejo indirecto.

8. Pila.h

Implementa una estructura de datos tipo Pila (stack).

Funcionalidad:

- Elementos insertar (push).
- Eliminar elementos (pop).
- Mostrar elementos.

Aplicamos estructura de datos y LIFO (ultimo en entrar, primero en salir).

9. ADT.h

Implementa un Tipo Abstracto de Datos (ADT).

Funcionalidad:

- Insertar datos.
- Eliminar datos.
- Buscar elementos.
- Mostrar contenido.

En este caso el usuario interactúa sin conocer la implementación interna.

10.Nodo.h

Define una estructura para nodos en una lista enlazada.

Funcionalidad:

- Almacena un dato.
- Apunta al siguiente nodo.

Aplicamos conceptos de punteros y estructuras auto referenciadas.

11.ListaPunteros.h

Implementa una lista enlazada simple.

Funcionalidad:

- Inserta nodos dinámicos.
- Recorre la lista.
- Muestra elementos.

Aplicamos memoria dinámica, punteros y estructuras enlazadas.

12.Main abs.cpp

Archivo principal que integra todo el sistema mediante un menú interactivo.

Funcionalidad:

- Permite seleccionar:
- Ordenamientos directos.
- Ordenamientos indirectos.
- Pila.
- ADT.
- Punteros.
- Ejecuta cada módulo.

Aplicamos modularidad, control fijo e interacción con el usuario.

13. Cola estática.

Implementar una estructura de datos tipo cola utilizando un arreglo, comprendiendo su funcionamiento bajo el principio FIFO y aplicando programación orientada a objetos.

Para esto se utilizo un arreglo junto con dos variables, utilizamos Enqueue y Dequeue.

En este caso la cola funciona correctamente respetando el orden FIFO y sin exceder su capacidad.

14. Lista estática.

El objetivo de este ejercicio es implementar una lista utilizando un arreglo, aplicando los principios de programación orientada a objetos y comprendiendo cómo se almacenan los datos de manera secuencial en memoria.

Una lista es una estructura de datos que permite almacenar elementos de manera ordenada. En una lista estática, los elementos se guardan en un arreglo de tamaño fijo, lo que implica una limitación en la capacidad.

Este tipo de estructura es útil cuando se conoce previamente el número máximo de elementos.

Para su implementación se utilizó un arreglo y una variable que controla el número de elementos almacenados. Se definieron las siguientes operaciones:

- Insertar (): agrega elementos al final del arreglo
- Eliminar (): reduce el tamaño de la lista
- Mostrar (): recorre el arreglo

Se utilizó una clase que implementa una interfaz (ADT), permitiendo mantener una estructura clara y organizada.

15.Pila dinámica.

Aquí implementamos una pila utilizando memoria dinámica, comprendiendo el comportamiento LIFO y el uso de punteros.

Una pila es una estructura de datos que sigue el principio LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir.

Las pilas dinámicas utilizan nodos enlazados, lo que permite crecer sin límite fijo.

Se utilizó una estructura tipo nodo con un puntero al siguiente elemento.

Operaciones:

- Push (): inserta en el tope
- Pop (): elimina el elemento superior
- Mostrar (): recorre la pila

16.Cola dinámica.

El objetivo fue Implementar una cola utilizando memoria dinámica para eliminar la limitación de tamaño.

Las colas dinámicas utilizan nodos enlazados, permitiendo un crecimiento flexible en memoria. También siguen el principio FIFO.

Se utilizó una estructura nodo con punteros:

frente: inicio de la cola.

fin: final de la cola.

- Enqueue ()
- Dequeue ()
- Mostrar ()

17. Lista dinámica.

Implementar una lista enlazada usando memoria dinámica.

Las listas dinámicas están formadas por nodos enlazados, lo que permite almacenar datos sin límite fijo.

Se utilizó una estructura nodo con un puntero al siguiente elemento.

Operaciones:

- Insertar ()
- Eliminar ()
- Mostrar ()

Se presentó un error al perder la referencia de los nodos al insertar.

Solución:

Se corrigió la asignación de punteros asegurando que cada nodo apunte correctamente al siguiente.

18. Pila STL.

Utilizar la librería estándar para implementar una pila de manera eficiente.

La STL proporciona estructuras ya implementadas como stack, facilitando el desarrollo.

Se utilizó la librería <stack> y sus funciones:

- Push ()
- Pop ()
- Top ()

19. Cola STL.

Implementar una cola usando la librería estándar.

La STL incluye queue, que facilita la implementación de colas.

Se utilizó <queue>:

- Push ()
- Pop ()
- Front ()

Fallo:

Se intentó acceder a front () cuando la cola estaba vacía.

Solución:

Se utilizó empty () para validar antes de acceder.

20. Lista STL.

Utilizar la librería list para manejar datos dinámicamente.

La STL ofrece listas doblemente enlazadas mediante list.

Se utilizó <list> con:

- push_back ()

Recorrido con iteradores

Fallo:

Se presentó un error al usar un ciclo for moderno incompatible con el compilador.

Solución:

Se utilizaron iteradores tradicionales.

ENLACE REPOSITORIO DE GITHUB:

<https://github.com/Alan1509A/estructuras-poo.git>

